

BANK_API · EXTENSION 5

API Design Best Practices

REST conventions, versioning, pagination, status codes and error handling, applied to the bank_api we have been building.

Prerequisite: Extension 4 solution (accounts, deposit / withdraw, transfer, KYC)

RECAP

Where bank_api stands after Extension 4

A working, layered FastAPI service - but with no rules yet for how it evolves.

Verb	Path	Purpose
POST	/accounts/	Open an account
GET	/accounts/	List accounts
GET	/accounts/{id}	Read one account
POST	/accounts/{id}/deposit	Add money
POST	/accounts/{id}/withdraw	Take money out
POST	/accounts/transfer	Move money A to B
GET	/accounts/{id}/kyc-status	Read KYC flag
PUT	/accounts/{id}/kyc-status	Set KYC flag

Router

HTTP in / out, status codes



Service

Business rules



Repository

SQL via SQLAlchemy



SQLite

bank.db

TODAY

What you will be able to do by the end

Five ideas, each one turned into a change to `bank_api` in the lab.

- 1 Why design matters** Treat an API as a contract that other teams depend on
- 2 REST conventions** Choose the right URL and HTTP verb, and know which calls are safe or idempotent
- 3 Versioning** Add `/api/v1` without breaking a single existing client
- 4 Big collections** Paginate, filter and sort transaction history
- 5 Status codes & errors** Return the right code and one consistent error body



1

Why API design matters

In banking, an API is how a mobile app, a web portal or a partner fintech touches customer money. Mistakes are expensive and hard to undo.

1 · WHY

An API is a contract

Once a client depends on a URL, a field name or a status code, you cannot change it without breaking them.

Security

Only the right caller sees or moves money.

bank_api: KYC check before a scheduled payment (403)

Reliability

The same request always behaves the same way.

bank_api: a retried transfer must not debit twice

Scalability

Responses stay small as data grows.

bank_api: transaction history is paginated

Developer experience

Clients can predict behaviour without reading your code.

bank_api: one error format everywhere under /api/v1

Design review: our own bank_api

Good bones - and five gaps we will close today without breaking anyone.

What is already good

- Plural nouns: /accounts
- Hierarchy: /accounts/{id}/kyc-status
- Layered: router, service, repository
- Pydantic validates every body

No version in the URL · Any change is a breaking change

Task 1

No transaction history · Balance changes leave no trail

Task 2

No way to page through data · One day a list returns 1 million rows

Task 3

Every error is {"detail": ...} · Clients cannot tell error types apart

Task 5

A retried transfer runs twice · A network blip can double-debit

Task 6

2

REST conventions

Resources are nouns. HTTP verbs are the actions. Status codes are the outcome.

Name resources, not actions

The URL says what you are working with. The HTTP verb says what you are doing to it.

Avoid	Prefer	Rule
GET /getAccount?id=7	GET /accounts/7	Nouns, not verbs
POST /createScheduledPayment	POST /accounts/7/scheduled-payments	The verb is already POST
GET /account/7	GET /accounts/7	Plural collection names
GET /Accounts/7/ScheduledPayments	GET /accounts/7/scheduled-payments	lowercase-with-hyphens
GET /transactions?account=7	GET /accounts/7/transactions	Nest when it belongs to a parent
DELETE /accounts/7/scheduled-payments/3	DELETE /scheduled-payments/3	Stop nesting once the id is unique

Pragmatic exception: some banking operations are commands (deposit, withdraw). POST /accounts/7/deposit is an accepted pattern for them.

2 · REST

The five verbs, applied to bank_api

Each verb has a meaning and a usual success code. Clients rely on both.

GET

Read. Never changes data.

GET /api/v1/accounts/7/transactions

200 OK

POST

Create something new (or run a command).

POST /api/v1/accounts/7/scheduled-payments

201 Created + Location

PUT

Replace the whole resource.

PUT /api/v1/accounts/7/kyc-status

200 OK

PATCH

Change only the fields you send.

PATCH /api/v1/scheduled-payments/3

200 OK

DELETE

Remove it.

DELETE /api/v1/scheduled-payments/3

204 No Content

PUT replaces, PATCH modifies

Same starting record, two very different results.

```
stored payment #3
{
  "payee_name": "Landlord",
  "amount": 25000,
  "frequency": "MONTHLY",
  "status": "ACTIVE"
}
```

```
PUT { "status": "PAUSED" }
{
  "payee_name": null,
  "amount": null,
  "frequency": null,
  "status": "PAUSED"
}
-> or 422: fields missing
```

```
PATCH { "status": "PAUSED" }
{
  "payee_name": "Landlord",
  "amount": 25000,
  "frequency": "MONTHLY",
  "status": "PAUSED"
}
```

In FastAPI: make every PATCH field Optional, then apply only `patch.model_dump(exclude_unset=True)`. That dict contains just the keys the client actually sent - so missing fields stay untouched.

Safe vs idempotent

Two properties that decide whether a client may retry a request on its own.

Verb	Safe	Idempotent	Retry blindly?
GET / HEAD	Yes	Yes	Yes
OPTIONS	Yes	Yes	Yes
PUT	No	Yes	Yes
DELETE	No	Yes	Yes (2nd call may 404)
POST	No	No	No - needs a key
PATCH	No	Not guaranteed	Only if designed so

Safe

Does not change server state. Read-only.

Idempotent

Sending it 1 time or 10 times leaves the server in the same state.

Every safe method is idempotent. Not every idempotent method is safe.

2 • CHECK

Quick check: safe? idempotent?

Discuss in pairs for 3 minutes. Answers on the next slide.

- A GET /api/v1/accounts/7
- B POST /api/v1/accounts/7/deposit {"amount": 500}
- C PUT /api/v1/accounts/7/kyc-status {"kyc_compliant": true}
- D DELETE /api/v1/scheduled-payments/3
- E PATCH /api/v1/scheduled-payments/3 {"status": "PAUSED"}
- F POST /api/v1/accounts/transfer + Idempotency-Key: abc-123

2 • CHECK

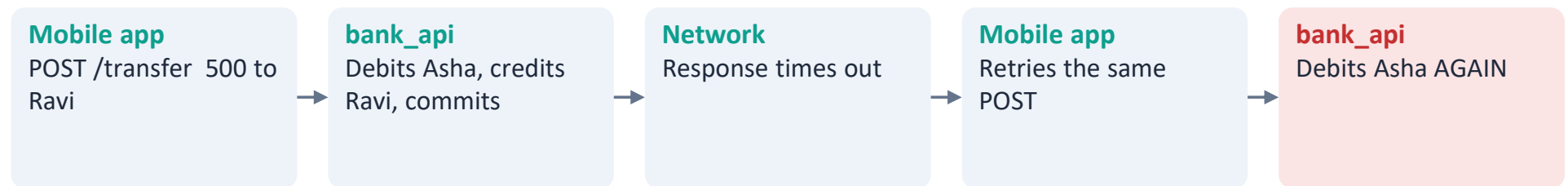
Answers

Idempotency is about the final state of the server, not about the response body.

	Safe	Idempotent	Why
A	Yes	Yes	Only reads.
B	No	No	Each call adds 500. Three retries = 1,500.
C	No	Yes	Sets the flag to true. The 10th call changes nothing.
D	No	Yes	Payment is gone after call 1. Call 2 returns 404, but the state is the same.
E	No	Yes	Sets status to PAUSED - an absolute value, not a relative change.
F	No	Yes, by design	Server remembers the key and replays the first response (Task 6).

Why this matters for money: the retry problem

The network, not your code, decides when a response gets lost.



```
The fix: an Idempotency-Key header
POST /api/v1/accounts/transfer
Idempotency-Key: 5f0c9e2a-...    <- new UUID per intent

1st call: key unseen -> transfer, store response
retry:    key seen   -> return stored response
```

Rules

- Client makes one key per user action
- Same key + same body = replay
- Same key + different body = 422
- Payment networks work this way



3

Versioning: change without breaking

Clients upgrade on their schedule, not yours. A version number lets old and new contracts live side by side.

Which changes break clients?

If an existing, correct client could fail after your deploy, it is a breaking change.

Safe to ship in v1 (additive)

- Add a new endpoint
- Add an optional query parameter
- Add a new field to a response
- Add an optional request header (Idempotency-Key)
- Accept a new enum value on input

Breaking - needs a new version

- Rename or remove a field (balance -> current_balance)
- Change a type (float -> string)
- Make an optional field required
- Change a status code clients check (200 -> 201)
- Change the error body shape ({detail} -> {error})

You will meet this in Task 5: the new error body is a breaking change for legacy clients, so it only applies under /api/v1. Old paths keep {"detail": ...}.

Three ways to version - we pick the URL path

All three work. The path wins on visibility, caching and ease of testing.

URL path

```
GET /api/v1/accounts/7
```

Visible in logs, links and browser.
Easy to route and cache.

Our choice

Header

```
Accept:  
application/vnd.bank.v1+json
```

Clean URLs, but invisible and harder
to test by hand.

Used by GitHub

Query param

```
GET /accounts/7?version=1
```

Easy to add, easy to forget. Mixes
routing with filters.

Least common

Rule of thumb: bump the major version only for breaking changes. Everything additive ships inside v1.

3 · VERSIONING

Implementing /api/v1 in two small steps

Wrap the existing router in a prefixed router. No endpoint code is copied.

```
NEW app/api/v1/__init__.py
from fastapi import APIRouter
from ...routers.account_router import (
    router as account_router)

router = APIRouter(prefix="/api/v1")
router.include_router(account_router)
```

```
EDIT app/main.py (+2 lines, 0 deleted)
from .api.v1 import router as v1_router

app.include_router(v1_router)      # new
app.include_router(account_router) # unchanged
```

legacy client
GET /accounts/7

new client
GET /api/v1/accounts/7

account_router
same code, same
database

Result: Both paths call the same function. Fix a bug once, both versions get it. /docs now lists both.

3 · VERSIONING

Retiring the old path politely

Keep /accounts working, but tell every legacy caller that it has an end date.

```
Response to GET /accounts/7 (legacy path)
HTTP/1.1 200 OK
Deprecation: @1790812800
Sunset: Thu, 31 Dec 2026 23:59:59 GMT
Link: </api/v1/accounts/7>; rel="successor-version"
X-Request-ID: req_3f9a1c0b77de
```

Header	Meaning
Deprecation	Since when (RFC 9745, Unix time)
Sunset	After this date it may stop (RFC 8594)
Link	Where to go instead

How: a small `@app.middleware("http")` adds these headers when `request.url.path` starts with `/accounts`. The body is untouched, so legacy clients keep working - but their monitoring and API gateways can see the warning.



4

Pagination, filtering & sorting

A savings account opened in 2015 can have 20,000 transactions. Nobody should download all of them to see last week's.

4 • COLLECTIONS

Anatomy of a collection request

Filters narrow the set, sort orders it, pagination slices it - always in that order.

```
GET  
/api/v1/accounts/7/transactions?type=WITHDRAWAL&min_amount=1000&sort_by=amount:desc&page=2&limit=10
```

1 Filter

WHERE clauses. Only for params the client sent. Multiple filters combine with AND.

2 Sort

ORDER BY a whitelisted column. Add id as a tie-breaker so pages never overlap.

3 Paginate

Count the filtered total, then OFFSET (page-1)*limit, LIMIT limit.

```
filter -> total = q.count() -> order_by(col, id) -> offset((page-1)*limit).limit(limit)
```

Return a page, not a bare list

The envelope tells the client where it is and how much is left.

```
200 OK
{
  "items": [ { "id": 91, "type": "WITHDRAWAL",
               "amount": 2500.0, ... } ],
  "page": 2,
  "limit": 10,
  "total": 37,
  "total_pages": 4
}
```

	Offset (page/limit)	Cursor (after=...)
Jump to page 7	Yes	No
Stable if rows are added	Can shift	Yes
Speed on huge tables	Slows down	Constant
Best for	Statements, admin UIs	Infinite scroll, feeds

Always: a default limit (25) and a hard maximum (100). Otherwise one client with limit=1000000 can overload the database for everyone.

Filtering and sorting rules that keep you safe

Every query parameter is untrusted input.

1

Whitelist sort fields

`sort_by=owner_name:asc` -> 400. Never pass user text to `getattr()` or raw SQL.

2

Type the parameters

type as an Enum, dates as date, limit with `ge=1`, `le=100`. FastAPI rejects bad values with 422.

3

Validate across fields

`min_amount > max_amount` or `from_date > to_date` -> 400 with a clear message.

4

Make date ranges inclusive

`to_date=2026-09-20` means the whole day: `created_at < 2026-09-21 00:00`.

5

Past the last page is fine

`page=99` returns 200 with items: `[]`. It is not an error.

4 • CHECK

Quick check: write the URL

Account 42. The 3rd page of its oldest withdrawals, 10 per page, only this September.

your answer

```
GET /api/v1/accounts/42/transactions?type=WITHDRAWAL&from_date=2026-09-01
    &to_date=2026-09-30&sort_by=created_at:asc&page=3&limit=10
```

Parameter	Kind	Why
type=WITHDRAWAL	Filter	Only withdrawals
from_date / to_date	Filter	Inclusive September range
sort_by=created_at:asc	Sort	Oldest first
page=3&limit=10	Pagination	Rows 21-30



5

Status codes & error handling

The status code tells a program what happened. The error body tells a human why - and what to do next.

The status codes bank_api actually uses

Learn these thirteen well. They cover almost every situation.

2xx Success

- 200** OK - here is the data
- 201** Created (+ Location)
- 204** No Content - empty body

4xx Client error

- 400** Bad Request - breaks a rule
- 401** Unauthorized - who are you?
- 403** Forbidden - known, not allowed
- 404** Not Found
- 405** Method Not Allowed
- 409** Conflict - clashes with state
- 422** Unprocessable - failed validation
- 429** Too Many Requests

5xx Server error

- 500** Internal Error - our bug
- 503** Unavailable - try later

Picking the right code: ask in this order

Stop at the first question that says no.

Is the request well-formed and valid?

no → **422 (FastAPI does this for you)**

Do we know who is calling?

no → **401**

Are they allowed to do this?

no → **403 e.g. KYC not complete**

Does the resource exist?

no → **404**

Does it obey the business rules?

no → **400 / 409 e.g. insufficient funds**

Did our own code fail?

no → **500 - log it, never leak it**

5 • ERRORS

One error shape for every /api/v1 failure

Clients write one error parser instead of one per endpoint.

```
400 Bad Request
{
  "error": {
    "code": "insufficient_funds",
    "message": "Insufficient funds",
    "details": { "current_balance": 54.2,
                  "requested_amount": 100.0 },
    "request_id": "req_36d6f19021ed"
  }
}
```

code

Stable, machine-readable. Clients branch on this.

message

For humans. Can change wording anytime.

details

Structured facts to act on.

request_id

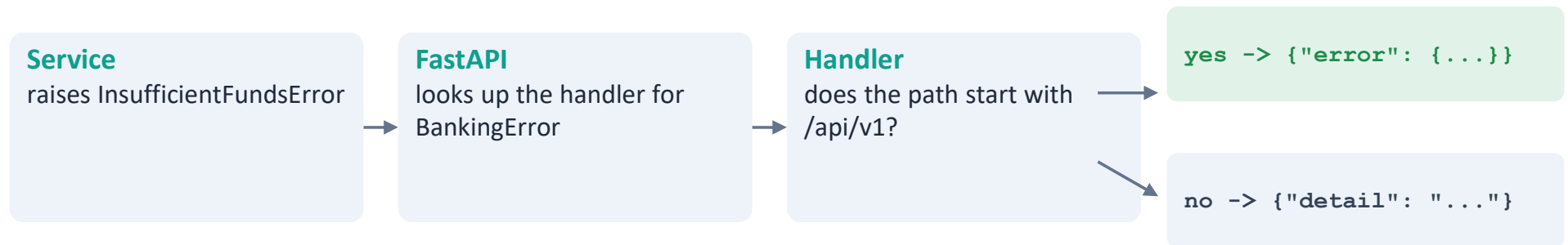
Also in the X-Request-ID header. Support finds the log line.

Never: stack traces, SQL, file paths or internal ids in an error body. Log them server-side against the request_id.

5 • ERRORS

Global exception handler: raise once, format once

Services raise domain errors. One handler turns every error into JSON.



```
app/core/errors.py (excerpt)
class BankingError(Exception):
    status_code = 400
    code = "bad_request"

class InsufficientFundsError(BankingError):
    code = "insufficient_funds"
```

Register four handlers

- BankingError (ours)
- HTTPException (404s, 405s)
- RequestValidationError (422)
- Exception (the 500 safety net)

WRAP-UP

Key takeaways

Good API design is empathy for the developer on the other side.

Be predictable

Nouns for resources, standard verbs, standard codes.

Plan for growth

Version from day one. Only additive changes inside a version.

Protect the server

Paginate every collection. Default 25, max 100.

Communicate clearly

One error envelope, stable codes, a request id.

Make retries safe

Idempotent by design - keys for money-moving POSTs.

Security first

Validate every input, whitelist sorts, never leak internals.

NEXT

Lab: extend bank_api (about 3 hours)

Lab guide = what to do. Rescue Kit deck = hints when you get stuck. Tests = proof it works.

1

Versioning + deprecation headers

20 min

2

Transaction ledger

25 min

3

Paginate, filter, sort transactions

40 min

4

Scheduled payments: 5 verbs, right codes

40 min

5

Global error handler + envelope

35 min

6

Stretch: Idempotency-Key on transfers

30 min

```
pytest tests/test_task1_versioning.py -v # run one task's tests at a time
```